

UNIVERSIDAD PRIVADA DEL VALLE

FACULTAD DE TECNOLOGÍA

INGENIERÍA DE SISTEMAS INFORMÁTICOS



SOFTWARE QUALITY ASSURANCE

Parcial 1

Integrantes:

- Osmer Suner Barrera Quiroga
- Carolina Cisneros Arroyo
- Fabián Castro Delgadillo
- Maximiliano Turdera Colque
- Jesús Eduardo Zeballos Isita

Docente: Ing. Juan Carlos Bergman

Fecha de entrega: 25-09-2026

SUCRE – BOLIVIA
2026

PLAN Y REPORTE DE PRUEBAS UNITARIAS Y FUNCIONALES (SQA)

Integrante	Rol Scrum	Módulo y Responsabilidad Técnica	Rama GitHub
Barrera Suner	Scrum Master / Integrador	Repositorio base, pipeline CI/CD, integración de Pull Requests y reporte global de cobertura JaCoCo.	chore/organizacion
Turdera Maximiliano	Developer / Tester	Módulo de Categorías (Service y Controller): casos exitosos, errores, 404, 500 y exportación.	test/categorias
Castro Fabian	Developer / Tester	Módulo de Productos (Service y Controller): CRUD, multipart, compresión ZLib e imágenes.	test/productos
Cisneros Carolina	Developer / Tester	Módulo de Utilidades: compresión ZLib, validación POI Excel y reporte formal de bugs.	test/utilidades
Zeballos Jesus	Developer / Tester	Pruebas Funcionales E2E en Postman/Newman, ejecución sobre MySQL en Docker y documentación.	test/postman-plan

1. INTRODUCCIÓN Y OBJETIVOS

1.1 Objetivo General

Diseñar, implementar y automatizar una suite integral de pruebas unitarias, de integración y funcionales sobre el proyecto proyecto-completo/inventory-sp3 basado en Spring Boot, aplicando aislamiento de dependencias con Mockito, validación de endpoints HTTP mediante MockMvc, pruebas funcionales End-to-End con Postman sobre base de datos MySQL en Docker y medición de cobertura de código con JaCoCo, bajo un marco colaborativo ágil (Scrum) gestionado en GitHub.

1.2 Objetivos Específicos

- Aislar la lógica de negocio simulando los repositorios DAOs mediante Mocks y verificar las interacciones críticas con verify y ArgumentCaptor.
- Alcanzar el 100% de cobertura de clases, métodos y líneas en las capas de Controladores, Servicios, Utilidades, Modelos y DTOs de Respuesta.
- Diseñar Historias de Usuario y Casos de Prueba tanto para flujos normales (HTTP 200 OK) como para flujos de error (HTTP 400 Bad Request, 404 Not Found y 500 Internal Server Error).
- Validar la integridad binaria de las imágenes comprimidas en ZLib y la estructura tabular de reportes Excel (.xlsx) generados con Apache POI.
- Registrar formalmente los defectos y riesgos técnicos en plantillas oficiales sin alterar la lógica de negocio de producción.

2. DEFINICIÓN DEL ALCANCE Y JUSTIFICACIÓN DE MÓDULOS

2.1 Módulos en Alcance

El alcance abarca 7 componentes nucleares distribuidos en las 3 capas requeridas por la consigna del proyecto:

Capa Arquitectural	Componentes Evaluados	Justificación Técnica de Selección
Controladores REST (com.company.inventory.controller)	CategoryRestController ProductRestController	Constituyen la interfaz externa de la API. Se debe validar el enlace HTTP, la deserialización de parámetros, la subida multipart de archivos y el mapeo correcto de códigos de estado sin levantar servidores reales.
Servicios de Negocio (com.company.inventory.services)	CategoryServiceImpl ProductServiceImpl	Albergan las reglas de negocio y transaccionales. Probar esta capa con Mockito asegura que el servicio responda adecuadamente ante cualquier respuesta del DAO (datos encontrados, listas vacías, nulos o excepciones de base de datos).
Utilidades y Reportes (com.company.inventory.util)	Util CategoryExcelExporter ProductExcelExporter	Manipulan flujos binarios en memoria (ZLib) y construyen libros XLSX. Un fallo en esta capa provocaría archivos corruptos o pérdida de imágenes de productos.

2.2 Componentes Fuera de Alcance

- Interfaz gráfica de usuario (frontend web o móvil).
- Pruebas de estrés, carga concurrente o seguridad de red.
- Modificación de la lógica de negocio original de producción.

3. ORGANIZACIÓN ÁGIL (SCRUM SIMPLIFICADO)

Se implementó un Sprint de dos días para estructurar, distribuir y coordinar el trabajo del equipo en GitHub.

3.1 Planificación del Sprint y Asignación de Tareas

Tarea (Issue)	Responsable	Actividades Día 1	Actividades Día 2	Criterio de Aceptación
Issue 1: Repositorio y CI	Suner Barrera	Crear repo base, invitar colaboradores, pipeline CI.	Integración de PRs, reporte JaCoCo global.	Suite completa aprobada en GitHub Actions.
Issue 2: Pruebas Categorías	Maximiliano Turdera	Crear tests unitarios Service y Controller.	Revisión cruzada de PR con Persona 3.	100% métodos cubiertos (200, 400, 404, 500).

Issue 3: Pruebas Productos	Fabian Castro	Tests unitarios Service, multipart y MockMvc.	Revisión cruzada de PR con Persona 2.	ArgumentCaptor verifica imagen y campos.
Issue 4: Utilidades y Bugs	Carolina Cisneros	Tests de compresión ZLib y lectura POI de Excel.	Redacción formal del reporte de bugs.	Inspección byte a byte y celdas XLSX validadas.
Issue 5: Postman y Reporte	Jesus Zeballos	Configurar contenedor MySQL y colección Postman.	Ejecución Newman y consolidación del informe.	25 requests con 57 assertions sin fallos.

3.2 Definición de Terminado (Definition of Done)

- Código de prueba implementado siguiendo el estándar JUnit 5 y Mockito.
- Ejecución local satisfactoria sin errores ni fallos (BUILD SUCCESS).
- Cobertura de código verificada en JaCoCo superior al 95%.
- Pull Request creado en GitHub con descripción detallada y referencia al Issue correspondiente.
- Revisión y aprobación realizada por otro integrante del equipo (Code Review cruzado).
- Integración exitosa en la rama principal (main).

TABLERO SCRUM KANBAN (ISSUES CERRADOS EN GITHUB)

The image displays two screenshots of the GitHub Issues interface. The top screenshot shows a list of four closed issues: 'Pruebas funcionales de Postman, plan e informe' (Issue #8), 'Pruebas de utilidades y exportación Excel' (Issue #7), 'test: agregar pruebas de categorías (servicio y REST)' (Issue #5), and 'test: agregar pruebas de productos (servicio y REST)' (Issue #3). The bottom screenshot shows another list of four closed issues: '[Max] Pruebas de servicio y controlador de categorías' (Issue #6), 'test: productos' (Issue #4), 'test: Carolina agrega pruebas de utilidades y exportacion Excel' (Issue #2), and 'test: entrega persona 5 - postman e informes' (Issue #1). Each issue entry includes the issue title, number, author, and status (opened/closed).

4. HISTORIAS DE USUARIO (HU)

Las Historias de Usuario fueron derivadas de los métodos expuestos por los controladores REST y redactadas bajo el estándar oficial con criterios de aceptación Gherkin (Dado / Cuando / Entonces):

Historia de Usuario	Descripción (Como / Quiero / Para)	Criterios de Aceptación (Gherkin)
HU01: Registro y Consulta de Categorías	Como encargado de inventario, quiero registrar nuevas categorías y consultar las existentes por ID o en listado general, para mantener clasificados los artículos del negocio.	Escenario 1 (Creación exitosa): Dado que se envía un nombre y descripción válidos, cuando se hace una petición POST a /api/v1/categories, entonces el sistema retorna HTTP 200 con el ID generado y metadato 00. Escenario 2 (Consulta ID inexistente): Dado un ID no registrado, cuando se hace GET a /api/v1/categories/{id}, entonces retorna HTTP 404 con metadato -1.
HU02: Actualización y Eliminación de Categorías	Como encargado de inventario, quiero modificar los datos de una categoría o darla de baja, para mantener el catálogo depurado.	Escenario 1 (Actualización exitosa): Dado un ID existente y nuevos valores, cuando se envía PUT a /api/v1/categories/{id}, entonces actualiza la entidad conservando su ID original y retorna HTTP 200. Escenario 2 (Eliminación): Dado un ID, cuando se envía DELETE a /api/v1/categories/{id}, entonces elimina el registro.
HU03: Exportación de Categorías a Excel	Como auditor de almacén, quiero descargar un reporte en Microsoft Excel con todas las categorías, para realizar inventarios físicos fuera de línea.	Escenario 1: Dado que existen categorías, cuando se solicita GET a /api/v1/categories/export/excel, entonces responde con HTTP 200, Content-Disposition: attachment; filename=result_category.xlsx y un archivo binario válido.
HU04: Registro de Productos con Imagen Multipart	Como encargado de almacén, quiero registrar un nuevo producto adjuntando su fotografía, precio, stock y categoría asociada, para catalogar los artículos visualmente.	Escenario 1 (Registro válido): Dado un formulario multipart/form-data con nombre, precio, cantidad, categoría existente e imagen, cuando se envía POST a /api/v1/products, entonces comprime la imagen en ZLib, persiste y retorna HTTP 200. Escenario 2 (Categoría no existe): Dado un categoryId inexistente, retorna HTTP 404. Escenario 3 (Datos inválidos): Dado un precio alfanumérico inválido (price='abc'), retorna HTTP 400 Bad Request.
HU05: Búsqueda y Filtrado de Productos	Como cajero o vendedor, quiero buscar productos por ID o filtrar por coincidencia de texto en el nombre, para localizar el stock y ver la imagen descomprimida.	Escenario 1 (Filtro por nombre): Dado un texto de búsqueda, cuando se realiza GET a /api/v1/products/filter/{name}, entonces retorna HTTP 200 con coincidencias e imagen descomprimida idéntica a la original. Escenario 2 (Sin coincidencias): Dado un término no existente, retorna HTTP 404.

HU06: Actualización y Retiro de Productos	Como administrador de inventario, quiero actualizar los datos y fotografía de un producto o eliminarlo, para reflejar cambios de precio o reabastecimiento.	Escenario 1: Dado un ID existente, nuevos datos e imagen, cuando se envía PUT multipart a /api/v1/products/{id}, se actualizan todos los campos conservando el ID y retorna HTTP 200.
HU07: Exportación de Productos a Excel	Como administrador de finanzas, quiero exportar la lista de productos con precio, cantidad y categoría a un libro Excel, para calcular el valor total del inventario.	Escenario 1: Dado que se realiza GET a /api/v1/products/export/excel, entonces descarga result_product.xlsx con cabeceras ID, Nombre, Precio, Cantidad y Categoría, con formato numérico en precio y cantidad.
HU08: Tratamiento Uniforme de Excepciones	Como desarrollador consumidor de la API, quiero recibir respuestas HTTP estandarizadas ante fallos del servidor, para manejar adecuadamente los errores.	Escenario 1 (Fallo de base de datos): Si el DAO lanza una excepción, el servicio retorna HTTP 500 con metadato -1 de forma controlada.

5. PLAN Y MATRIZ DE CASOS DE PRUEBA

El diseño del plan de pruebas se encuentra formalizado en el documento adjunto **Plan_de_Pruebas_Inventory_Final.docx** el cual especifica el alcance de las 8 Historias de Usuario HU01-HU08 en formato Gherkin, ambientes y matriz de riesgos.

Anexo A - Matriz Excel Oficial (Casos_de_Prueba_Inventory.xlsx): La suite de pruebas abarca **114 casos detallados** (89 unitarios/integración + 25 funcionales E2E en Postman) registrados en la plantilla oficial:

- **Archivo Anexo A:** Casos_de_Prueba_Inventory.xlsx (Contiene los 114 casos con precondiciones, pasos 1 a 4, datos de entrada y resultados en estado PASÓ).

5.1 Resumen de Pruebas Unitarias e Integración (JUnit 5, Mockito y MockMvc)

Se ejecutaron 89 métodos de prueba unitaria e integración con resultado **BUILD SUCCESS**:

- **Capa de Categorías (21 casos):** Cobertura del 100% en CategoryServiceImplTest, CategoryRestControllerTest y CategoryControllerAdditionalTest.
- **Capa de Productos (37 casos):** Cobertura del 100% en ProductServiceImplTest y ProductRestControllerTest probando cargas multipart e imágenes ZLib.
- **Capa de Utilidades y Reportes (9 casos):** Cobertura en UtilTest, UtilAdditionalTest y ExcelExportersTest para compresión ZLib y exportación Excel.
- **Modelos, Respuestas y DAOs (17 casos):** Cobertura en CategoryTest, ProductTest, ResponseRestTest, ICategoryDaoTest e IProductDaoTest.

- **Clase Principal (5 casos):** Carga del contexto Spring Boot e inicialización de beans.

Cover inventory in inventory-sp3

✓ missingCategoryDoesNotSave() 2ms
 ✓ updateSaveFailureReturns500() 3ms
 ✓ findsAndDecompressesImage() 1ms
 ✓ deletesRequestedId() 1ms
 ✓ categoryLookupExceptionReturns500() 1ms
 ✓ saveExceptionReturns500() 1ms
 ✓ findExceptionReturns500() 2ms
 ✓ filtersByNameAndDecompresses() 3ms
 ✓ updateMissingProductDoesNotSave() 2ms
 ✓ savesWithCategory() 3ms
 ✓ updatesEveryFieldAndKeepsId() 3ms
 ✓ nullSaveReturns400() 3ms
 ✓ filterFailureReturns500() 1ms
 ✓ noNameMatchesReturns404() 2ms
 ✓ updateCategoryFailureReturns500() 2ms
 ✓ missingProductReturns404() 2ms
 ✓ emptyListReturns404() 2ms
 ✓ listFailureReturns500() 3ms
 ✓ deletionFailureReturns500() 2ms
 ✓ listsAndDecompressesAllPictures() 2ms

98 tests passed 98 tests total, 22 sec 581 ms

Compressed Image Byte Size - 12
 Compressed Image Byte Size - 12
 Compressed Image Byte Size - 12
 Compressed Image Byte Size - 12
 Compressed Image Byte Size - 12
 Compressed Image Byte Size - 12
 2026-09-25T16:00:50.037-04:00 INFO 47052 --- [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Clos
 2026-09-25T16:00:50.037-04:00 INFO 47052 --- [ionShutdownHook] .SchemaDropperImpl\$DelayedDropActionImpl : HHH0
 2026-09-25T16:00:50.039-04:00 DEBUG 47052 --- [ionShutdownHook] org.hibernate.SQL : drop
 Hibernate: drop table if exists category cascade
 2026-09-25T16:00:50.043-04:00 DEBUG 47052 --- [ionShutdownHook] org.hibernate.SQL : drop
 Hibernate: drop table if exists product cascade
 2026-09-25T16:00:50.052-04:00 INFO 47052 --- [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Clos
 2026-09-25T16:00:50.052-04:00 INFO 47052 --- [ionShutdownHook] .SchemaDropperImpl\$DelayedDropActionImpl : HHH0
 2026-09-25T16:00:50.052-04:00 DEBUG 47052 --- [ionShutdownHook] org.hibernate.SQL : drop
 2026-09-25T16:00:50.054-04:00 DEBUG 47052 --- [ionShutdownHook] org.hibernate.SQL : drop
 2026-09-25T16:00:50.055-04:00 INFO 47052 --- [ionShutdownHook] com.zaxxer.hikari.HikariDataSource : Hikari
 2026-09-25T16:00:50.058-04:00 INFO 47052 --- [ionShutdownHook] com.zaxxer.hikari.HikariDataSource : Hikari

Process finished with exit code 0

5.2 Resumen de Pruebas Funcionales End-to-End

SQA examen - Categorías y P... - Run results

Ran today at 02:43:00 PM · View all runs

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	Inventory local	1	2s 598ms	57	0	9 ms

All 57 Passed 57 Failed 0 Skipped 0 Errors 0 Console log

Iteration 1

POST 01 Crear categoría propia
 http://localhost:8080/api/v1/categories
 PASS HTTP 200
 PASS Código de negocio
 PASS Categoría creada

GET 02 Listar categorías
 http://localhost:8080/api/v1/categories
 PASS HTTP 200
 PASS Código de negocio
 PASS Contiene categoría creada

GET 03 Consultar categoría
 http://localhost:8080/api/v1/categories/5
 PASS HTTP 200
 PASS Código de negocio
 PASS Nombre persistido

PUT 04 Actualizar categoría
 http://localhost:8080/api/v1/categories/5
 PASS HTTP 200
 PASS Código de negocio
 PASS Categoría actualizada

GET 05 Verificar actualización categoría
 http://localhost:8080/api/v1/categories/5
 PASS HTTP 200
 PASS Cambio persistente

Se ejecutaron los **25 casos funcionales E2E (CF-01 a CF-25)** en Postman contra el backend local logrando **100% de éxito** en sus 57 afirmaciones (assertions):

- **Flujos CRUD Categorías y Productos (CF-01 a CF-11):** Alta, consulta, filtro por nombre e imágenes Base64.

- **Exportación Excel (CF-12 y CF-13):** Descarga binaria de result_category.xlsx y result_product.xlsx.
- **Flujos Negativos y Control de Errores (CF-14 a CF-21):** Validación de 404 Not Found, 400 Bad Request y metadatos de error (-1).
- **Eliminación Segura (CF-22 a CF-25):** Borrado de registros y confirmación de baja definitiva.

Se ejecutó la colección automatizada de 25 peticiones HTTP sobre el contenedor Docker MySQL con 57 aserciones sin fallos (casos CF-01 a CF-12).

6. HERRAMIENTAS Y RESULTADOS DE COBERTURA (JACOCO)

6.1 Herramientas Empleadas

- **Lenguaje y Framework:** Java 17 (LTS), Spring Boot 3.1.0, Spring Data JPA.
- **Testing Unitario:** JUnit 5 (Jupiter), Mockito 5 (@Mock, @InjectMocks, ArgumentCaptor).
- **Testing de Controladores:** Spring Test MockMvc (Standalone Setup).
- **Testing Funcional E2E:** Postman Desktop y Newman CLI sobre contenedor Docker MySQL.
- **Métricas de Cobertura:** JaCoCo Maven Plugin versión 0.8.13.

6.2 Resultados Cuantitativos de Cobertura de Código

Comando ejecutado: .\mvnw.cmd -B clean verify | Total pruebas: 98 | Fallos: 0 | Errores: 0 | Omitidas: 0 | Estado: BUILD SUCCESS (100% Aprobadas)

Coverage inventory in inventory-sp3				
Element ^	Class, %	Method, %	Line, %	Branch, %
✓ com.company.inventory	100% (15/15)	100% (62/62)	100% (336/336)	100% (134/134)
> controller	100% (2/2)	100% (14/14)	100% (50/50)	100% (0/0)
> dao	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
> model	100% (2/2)	100% (13/13)	100% (13/13)	100% (66/66)
> response	100% (5/5)	100% (8/8)	100% (12/12)	100% (28/28)
> services	100% (2/2)	100% (14/14)	100% (164/164)	100% (24/24)
> util	100% (3/3)	100% (12/12)	100% (96/96)	100% (16/16)
InventorySp3Application	100% (1/1)	100% (1/1)	100% (1/1)	100% (0/0)

Nota Técnica: Se alcanzó el 100.00% de cobertura de clases, métodos y líneas en todas las capas del proyecto (Controladores, Servicios, Utilidades, Modelos y Respuestas), registrando un 99.00% global.

REPORTES DE COBERTURA DE CÓDIGO JACOCO EN NAVEGADOR WEB

inventory-sp3

inventory-sp3

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.company.inventory.util		100 %		100 %	0 21	2 120	0 13	0 3
com.company.inventory.services		100 %		100 %	0 27	0 193	0 15	0 2
com.company.inventory.model		100 %		100 %	0 62	0 13	0 29	0 2
com.company.inventory.response		100 %		100 %	0 37	0 19	0 23	0 5
com.company.inventory.controller		100 %	n/a	n/a	0 15	0 54	0 15	0 2
com.company.inventory		100 %	n/a	n/a	0 2	0 3	0 2	0 1
Total	3 of 2.072	99 %	0 of 134	100 %	0 164	2 402	0 97	0 15

7. CONCLUSIONES

- **Cumplimiento Integral de Objetivos:** Se desarrolló y ejecutó una suite de 98 pruebas unitarias/integración con 0 fallos, alcanzando el 100% de cobertura en clases, métodos y líneas en las capas núcleo y un 99.00% global.
- **Efectividad del Mocking:** El uso de Mockito y MockMvc permitió evaluar con precisión el comportamiento de controladores y servicios ante fallos simulados de persistencia (404 y 500) y entradas inválidas (400), sin depender de una base de datos activa para las pruebas unitarias.
- **Validación Funcional Complementaria:** La colección de Postman con 25 solicitudes y 57 validaciones sobre MySQL garantizó que la persistencia real, subida de imágenes y exportación de archivos funcione perfectamente en un entorno integrado.
- **Colaboración Ágil:** El flujo de trabajo en GitHub mediante ramas por integrante, Pull Requests y trazabilidad con issues permitió evidenciar el aporte individual y la integración continua del equipo.